Task 2 Tournament Ranking with Time Control

105.173 Bayesian Statistics

Philipp Schott

2026-11-11

Contents

Load Data **2**

Task 2 **2**

Collecting Ideas 2

Continue with Idea 2 2

Results **4**

Load Data

```
# Load dataset
games <- read.csv("../../../data/games.csv")

# Basic characteristics
str(games)
```

```
## 'data.frame':    862 obs. of  4 variables:
##  $ white      : chr  "A" "SM" "SM" "SM" ...
##  $ black      : chr  "SM" "A" "A" "A" ...
##  $ result     : chr  "1-0" "1/2-1/2" "1/2-1/2" "1-0" ...
##  $ timecontrol: chr  "60+1" "180+1" "60+1" "60+2" ...
```

```
cat("Missing values:" , sum(is.na(games)))
```

```
## Missing values: 0
```

Task 2

Collecting Ideas

When reading the assignment I have 2 main ideas:

- Idea 1: we assume that different time controls represent different game modes, therefore different distributions in rating. My first idea would be to model a hierarchical model capturing the different populations of games assuming a general underlying player strength.
- Idea 2: Besides the general underlying player strength players could have strength in different game modes eg., R1 $>$ R2 for one game mode but R1 $<$ R2 in a different game mode.

Continue with Idea 2

By further reading, I think Idea 2 is expected one. Therefore I will continue with that one as we by the note we assume linear dependence of timecontrol, ie.:

$$\boxed{R_j(t_c) = \alpha_j + \beta_j x(t_c)}$$

where $x(t_c)$ is the z-scaled game duration.

- α_j: Base strength of engine j, modeled in exactly the same way as R_j in Task 1 (with draws).
- β_j: How its strength changes with thinking time, with

$$\beta_j \sim \mathcal{N}(0, 100^2).$$

- $t_c = \text{time} + 40 \cdot \text{increment}$: time as a numeric variable.

Data Preparation for Stan

```r
# Engine identifiers
engines <- sort(unique(c(games$white, games$black)))

# Convert time controls into expected game duration
parts <- strsplit(games$timecontrol, "\\+")

tc <- vapply(parts, function(p) {
  as.numeric(p[1]) + 40 * as.numeric(p[2])
}, numeric(1))

# Standardize time control
tc_center <- mean(tc)
tc_scale <- sd(tc)
stopifnot(is.finite(tc_scale), tc_scale > 0)

x <- (tc - tc_center) / tc_scale

# Encode the three game outcomes
outcome_levels <- c("1-0", "1/2-1/2", "0-1")

# Data for the time-control model
stan_data_tc <- list(
  N = nrow(games),
  K = length(engines),
  white = match(games$white, engines),
  black = match(games$black, engines),
  outcome = match(games$result, outcome_levels),
  x = as.numeric(x)
)
```

Posterior Sampling

```r
# Compile time-control model
mod_tc <- cmdstan_model("helpers/rating_timecontrol.stan")

# Sample from the posterior
fit_tc <- mod_tc$sample(
  data = stan_data_tc,
  seed = 42
)
```

Table 1: Time-Control Model: Posterior Summary and Diagnostics

variable	mean	sd	rhat	ess_bulk	ess_tail
intercept[1]	2042.293	86.813	1.002	985.010	1792.052
intercept[2]	1905.841	90.816	1.002	1099.791	1866.966
intercept[3]	1941.550	90.667	1.002	1049.309	1912.785
intercept[4]	1792.369	86.615	1.003	995.979	1711.664
intercept[5]	2299.564	85.845	1.003	954.714	1809.962
intercept[6]	2030.140	84.257	1.003	947.626	1704.270

variable	mean	sd	rhat	ess_bulk	ess_tail
slope[1]	-14.940	52.175	1.001	2328.680	2604.460
slope[2]	-38.272	62.447	1.002	2653.707	2762.920
slope[3]	-7.708	61.464	1.000	2726.009	2861.140
slope[4]	47.933	53.165	1.001	2239.895	2283.780
slope[5]	24.488	45.693	1.002	1890.554	2370.533
slope[6]	-7.530	44.679	1.001	1852.266	2399.396
draw_tendency	0.298	0.031	1.003	3467.281	2635.278

The MCMC diagnostics indicate satisfactory sampling performance. All R-hat values are close to 1, all bulk effective sample sizes exceed 1,000 and all tail effective sample sizes exceed 100. Thus, the sampling diagnostics do not indicate convergence problems.

Results

Table 2: Engine Ranking at Average Time Control

Engine	mean	sd
E	2299.6	85.8
A	2042.3	86.8
SM	2030.1	84.3
C	1941.5	90.7
B	1905.8	90.8
D	1792.4	86.6

The estimated ratings at the average time control are very similar to those obtained in Task 1, yielding the same ranking: E > A > SM > C > B > D. Posterior standard deviations are also comparable. However, the time-dependent model additionally allows relative engine strengths to change across different time controls.

```
# Extract posterior samples for base ratings and time effects
alpha <- as.matrix(fit_tc$draws(
  variables = "intercept", format = "draws_matrix"
))

beta <- as.matrix(fit_tc$draws(
  variables = "slope", format = "draws_matrix"
))

# Calculate engine ratings at a given time control
rating_at_time <- function(t) {
  x <- (t - tc_center) / tc_scale
  alpha + beta * x
}
```

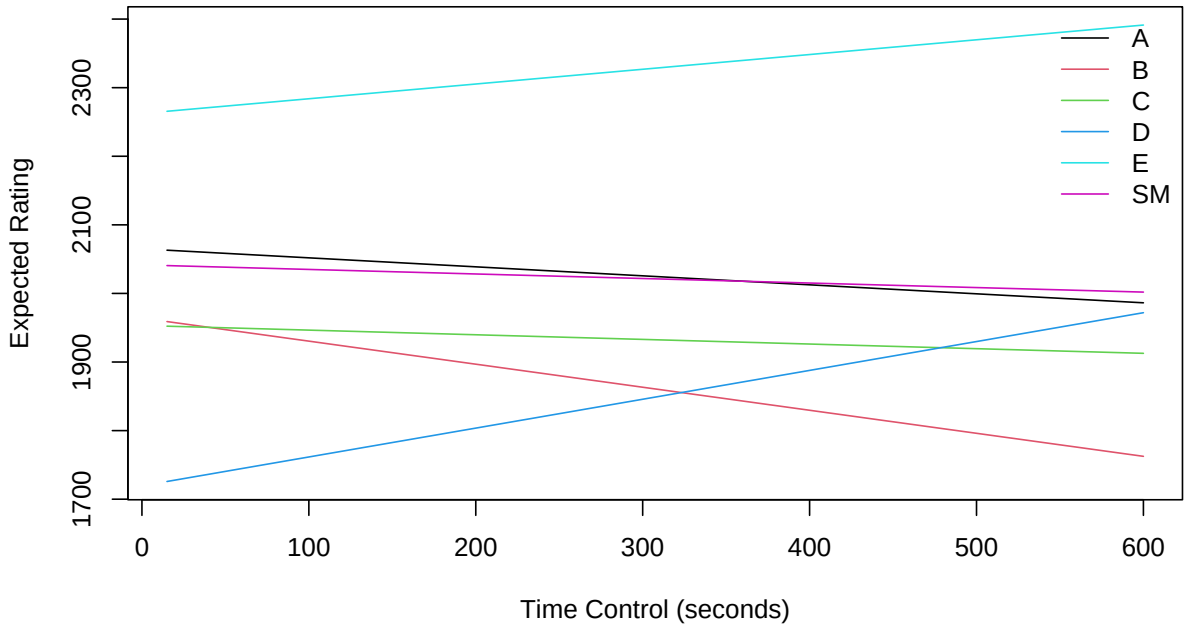
We visualize the posterior mean ratings across different time controls to examine how engine strength changes with game duration.

```r
# Define time controls from 15 to 600 seconds
times <- seq(15, 600, length.out = 100)

# Calculate expected ratings for each engine and time control
ratings <- t(sapply(times, function(t) {
  colMeans(rating_at_time(t))
}))

# Plot time-dependent ratings for all engines
matplot(times, ratings, type = "l", lty = 1,
        xlab = "Time Control (seconds)",
        ylab = "Expected Rating")

legend("topright", legend = engines,
       col = 1:6, lty = 1, bty = "n")
```



The plot shows that Engine E remains the strongest, while the other engines exhibit different time-dependent trends, motivating separate tier lists for the four specified time controls.

```r
# Rank engines based on their ratings at a given time control
tier_ranking <- function(t) {
  draws <- rating_at_time(t)
  means <- colMeans(draws)
  idx <- order(-means)

  # Calculate superiority probabilities between neighboring engines
  p <- sapply(1:(length(engines) - 1), function(i) {
```

```
    mean(draws[, idx[i]] > draws[, idx[i + 1]])
  })

  # Assign a new tier if superiority probability exceeds 95%
  data.frame(
    Tier = c(1, 1 + cumsum(p >= 0.95)),
    Engine = engines[idx],
    Mean = means[idx],
    SD = apply(draws, 2, sd)[idx]
  )
}
```

Table 3: Tier List: 15+0

	Tier	Engine	Mean	SD
intercept[5]	1	E	2265.6	107.0
intercept[1]	2	A	2063.0	112.9
intercept[6]	2	SM	2040.6	105.3
intercept[2]	2	B	1959.0	121.8
intercept[3]	2	C	1952.3	126.4
intercept[4]	3	D	1725.8	114.3

Table 4: Tier List: 60+0

	Tier	Engine	Mean	SD
intercept[5]	1	E	2275.2	97.3
intercept[1]	2	A	2057.1	101.0
intercept[6]	2	SM	2037.6	95.7
intercept[3]	2	C	1949.2	110.8
intercept[2]	2	B	1943.9	106.9
intercept[4]	3	D	1744.7	101.8

Table 5: Tier List: 180+2

	Tier	Engine	Mean	SD
intercept[5]	1	E	2318.2	92.5
intercept[1]	2	A	2030.9	95.6
intercept[6]	2	SM	2024.4	90.5
intercept[3]	2	C	1935.7	100.8
intercept[2]	2	B	1876.7	105.0
intercept[4]	2	D	1828.9	95.3

Table 6: Tier List: 600+0

	Tier	Engine	Mean	SD
intercept[5]	1	E	2391.3	191.1
intercept[6]	2	SM	2001.9	186.4

	Tier	Engine	Mean	SD
intercept[1]	2	A	1986.3	214.1
intercept[4]	2	D	1971.9	216.5
intercept[3]	2	C	1912.7	244.9
intercept[2]	2	B	1762.5	255.9

Engine E consistently occupies the highest tier, while the remaining rankings change with time control. Particularly at 600+0, increased posterior uncertainty prevents a clear separation of the other engines under our 95% superiority criterion.